

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA5112

COURSE OUTCOME COVERED

CO3: Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

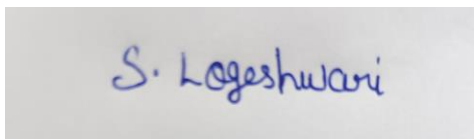
TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

I Logeshwari S (192565040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

EXPERIMENT 1: PASSWORD HASHING WITH SALTING AND KEY STRETCHING

QUESTION

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

ANSWER

AIM

To implement secure password storage and verification using PBKDF2-HMAC-SHA256 with a unique salt and key stretching.

SOFTWARE / REQUIREMENTS

Python 3.x; hashlib; hmac; os; time.

ALGORITHM

1. Generate a secure random salt.
2. Apply PBKDF2-HMAC-SHA256 with a high iteration count.
3. Store salt, iterations and derived hash.
4. Recompute the derived hash during verification.
5. Compare the values securely.

PYTHON CODE

Experiment 1 - Python Code

```
import os
import hashlib
import hmac
import time

def hash_password(password):
    salt = os.urandom(16)
    iterations = 200000
    start = time.perf_counter()
    key = hashlib.pbkdf2_hmac(
        "sha256", password.encode(), salt, iterations)
    elapsed = time.perf_counter() - start
    return salt, key, iterations, elapsed

def verify_password(password, salt, stored_key, iterations):
    key = hashlib.pbkdf2_hmac(
        "sha256", password.encode(), salt, iterations)
    return hmac.compare_digest(key, stored_key)

password = input("Enter password: ")
salt, stored_key, iterations, t = hash_password(password)

print("Salt:", salt.hex())
print("Stored Hash:", stored_key.hex())
print("Iterations:", iterations)
print("Hashing Time: %.4f seconds" % t)

check = input("Re-enter password: ")
print("Verification:", "SUCCESS" if verify_password(
    check, salt, stored_key, iterations) else "FAILED")
```

INPUT & OUTPUT

Experiment 1 - Input / Output

Enter password: Secure@123

Re-enter password: Secure@123

Sample output:

Salt: <random 16-byte hexadecimal value>

Stored Hash: <64-character hexadecimal derived key>

Iterations: 200000

Hashing Time: <measured on the system>

Verification: SUCCESS

ANALYSIS

The program uses a unique random salt for every password and PBKDF2-HMAC-SHA256 for key stretching. Salting prevents identical passwords from producing the same stored value and makes precomputed rainbow tables impractical. The high iteration count deliberately increases the cost of password guessing. The exact execution time depends on the computer used.

RESULT

The password was successfully hashed, stored, and verified using salting and key stretching. The experiment demonstrates stronger protection against brute-force and rainbow-table attacks.

EXPERIMENT 2: IMPLEMENTATION OF HMAC FOR MESSAGE INTEGRITY

QUESTION

Develop a Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

ANSWER

AIM

To implement HMAC using SHA-256 and verify message integrity and shared-secret-based authenticity.

SOFTWARE / REQUIREMENTS

Python 3.x; hmac; hashlib.

ALGORITHM

1. Read the message and secret key.
2. Generate HMAC-SHA256.
3. Display the tag. 4. Recalculate the tag for the received message.
5. Compare both tags.
6. Accept only when they match.

PYTHON CODE

Experiment 2 - Python Code

```
import hmac
import hashlib

key = input("Enter secret key: ").encode()
message = input("Enter message: ").encode()

tag = hmac.new(key, message, hashlib.sha256).hexdigest()
print("HMAC-SHA256:", tag)

received = input("Enter received message: ").encode()
new_tag = hmac.new(key, received, hashlib.sha256).hexdigest()

if hmac.compare_digest(tag, new_tag):
    print("Result: AUTHENTIC AND INTEGRITY VERIFIED")
else:
    print("Result: MESSAGE MODIFIED OR AUTHENTICATION FAILED")
```

INPUT & OUTPUT

Experiment 2 - Input / Output

```
Enter secret key: secret123
Enter message: Transfer Rs.1000
Enter received message: Transfer Rs.1000

Sample output:
HMAC-SHA256: <64-character hexadecimal authentication tag>
Result: AUTHENTIC AND INTEGRITY VERIFIED
```

ANALYSIS

A simple hash does not use a secret key, so a modified message can have a newly calculated hash. HMAC uses a shared secret key with SHA-256, allowing the receiver to verify both message integrity and shared-secret-based authenticity. HMAC does not provide non-repudiation.

RESULT

The program successfully generated and verified an HMAC-SHA-256 tag. The message was accepted because the calculated tags matched.

EXPERIMENT 3: SIMULATION OF CERTIFICATE CHAIN VALIDATION

QUESTION

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

ANSWER

1. AIM

To simulate validation of an X.509 certificate chain from an end-entity certificate through an intermediate CA to a trusted root CA.

2. SOFTWARE / REQUIREMENTS

Python 3.x. Simplified certificate objects are used to demonstrate chain-validation logic.

3. ALGORITHM

1. Create a trusted root CA.
2. Create an intermediate CA issued by the root.
3. Create an end-entity certificate issued by the intermediate.
4. Check validity and revocation.
5. Check issuer relationships.
6. Confirm the trusted root.

PYTHON CODE

Experiment 3 - Python Code

```
class Certificate:
    def __init__(self, subject, issuer, valid=True, revoked=False):
        self.subject = subject
        self.issuer = issuer
        self.valid = valid
        self.revoked = revoked

root = Certificate("RootCA", "RootCA")
intermediate = Certificate("IntermediateCA", "RootCA")
server = Certificate("www.example.com", "IntermediateCA")

chain = [server, intermediate, root]

def validate_chain(chain):
    for i, cert in enumerate(chain):
        if not cert.valid:
            return False, cert.subject + " is invalid"
        if cert.revoked:
            return False, cert.subject + " is revoked"
        if i < len(chain) - 1:
            parent = chain[i + 1]
            if cert.issuer != parent.subject:
                return False, "Invalid issuer relationship"

    if chain[-1].subject != chain[-1].issuer:
        return False, "Untrusted root"
    return True, "Certificate chain is valid and trusted"

ok, msg = validate_chain(chain)
print(msg)
```

INPUT & OUTPUT

Experiment 3 - Input / Output

```
Input:
Root CA = RootCA
Intermediate CA = IntermediateCA
End Entity = www.example.com
Status = valid and not revoked

Output:
Certificate chain is valid and trusted
```

ANALYSIS

Trust is anchored at a trusted root CA. The end-entity certificate is accepted only when the issuer relationships form a valid chain to that root. A real X.509 implementation additionally checks signatures, validity periods, key usage, constraints, and revocation through mechanisms such as CRL or OCSP. This experiment is a simplified simulation.

RESULT

The simulation successfully validated the certificate chain and demonstrated rejection conditions for invalid, revoked, or incorrectly issued certificates.

EXPERIMENT 4: REPLAY ATTACK SIMULATION AND PREVENTION

QUESTION

Develop a Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

ANSWER

AIM

To demonstrate a replay attack and prevent repeated use of an authentication request using a unique nonce.

SOFTWARE / REQUIREMENTS

Python 3.x; secrets; time.

ALGORITHM

1. Generate a unique nonce.
2. Store it after successful acceptance.
3. Submit the same request again.
4. Check whether the nonce was already used.
5. Reject the repeated request.

PYTHON CODE

Experiment 4 - Python Code

```
import secrets
import time

nonce = secrets.token_hex(8)
timestamp = int(time.time())
used_nonces = set()

print("Generated Nonce:", nonce)
print("Timestamp:", timestamp)

def accept_request(n):
    if n in used_nonces:
        print("Replay detected: request rejected")
    else:
        used_nonces.add(n)
        print("Request accepted")

print("\nFirst request:")
accept_request(nonce)

print("\nReplay request:")
accept_request(nonce)
```

5. INPUT & OUTPUT

Experiment 4 - Input / Output

Input:
A unique nonce is generated automatically.
The same nonce is submitted twice to simulate replay.

Sample output:
Generated Nonce: <random hexadecimal value>
Timestamp: <current Unix timestamp>

First request:
Request accepted

Replay request:
Replay detected: request rejected

ANALYSIS

A replay attack reuses a previously valid authentication message. The nonce provides freshness because it is intended for one-time use. Timestamp-based prevention can also

limit the lifetime of a request, but requires reasonable clock synchronization. Nonce tracking requires appropriate server-side state or a challenge-response design.

RESULT

The experiment successfully detected and rejected the replayed request because the nonce had already been used.

EXPERIMENT 5: PERFORMANCE COMPARISON OF DIGITAL SIGNATURE ALGORITHMS

QUESTION

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

ANSWER

AIM

To generate, sign, verify, and compare RSA and ECDSA digital signatures using measured execution time.

SOFTWARE / REQUIREMENTS

Python 3.x; cryptography library. Install with: `pip install cryptography`.

ALGORITHM

1. Generate RSA-2048 and ECDSA P-256 key pairs.
2. Sign the same message.
3. Verify both signatures.
4. Measure key-generation, signing and verification time.
5. Compare the algorithms.

PYTHON CODE

Experiment 5 - Python Code

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, ec, padding
import time

message = b"Secure message"

# RSA-2048
t = time.perf_counter()
rsa_private = rsa.generate_private_key(
    public_exponent=65537, key_size=2048)
rsa_key_time = time.perf_counter() - t
rsa_public = rsa_private.public_key()

t = time.perf_counter()
rsa_sig = rsa_private.sign(
    message, padding.PKCS1v15(), hashes.SHA256())
rsa_sign_time = time.perf_counter() - t

t = time.perf_counter()
rsa_public.verify(
    rsa_sig, message, padding.PKCS1v15(), hashes.SHA256())
rsa_verify_time = time.perf_counter() - t

# ECDSA P-256
t = time.perf_counter()
ec_private = ec.generate_private_key(ec.SECP256R1())
ec_key_time = time.perf_counter() - t
ec_public = ec_private.public_key()

t = time.perf_counter()
ec_sig = ec_private.sign(message, ec.ECDSA(hashes.SHA256()))
ec_sign_time = time.perf_counter() - t

t = time.perf_counter()
ec_public.verify(ec_sig, message, ec.ECDSA(hashes.SHA256()))
ec_verify_time = time.perf_counter() - t

print("Algorithm  KeyGen(s)  Sign(s)  Verify(s)")
print("RSA-2048      %.6f    %.6f    %.6f" %
      (rsa_key_time, rsa_sign_time, rsa_verify_time))
print("ECDSA-P256   %.6f    %.6f    %.6f" %
      (ec_key_time, ec_sign_time, ec_verify_time))
print("Both signatures verified successfully.")
```

INPUT & OUTPUT

Experiment 5 - Input / Output

Input:

Message = Secure message

RSA key size = 2048 bits

ECDSA curve = SECP256R1 (P-256)

Note: The code below contains the corrected verification call used in the final program:

```
ec_public.verify(ec_sig, message, ec.ECDSA(hashes.SHA256()))
```

Sample output:

Algorithm	KeyGen(s)	Sign(s)	Verify(s)
RSA-2048	<measured>	<measured>	<measured>
ECDSA-P256	<measured>	<measured>	<measured>

Both signatures verified successfully.

ANALYSIS

Exact timing values depend on the computer, Python version, operating system, and cryptographic library, so they should be measured by executing the program. RSA offers broad compatibility and efficient verification with common public-exponent choices. ECDSA uses much smaller keys and signatures for comparable security and is attractive where storage and bandwidth are constrained.

RESULT

The experiment demonstrates successful key generation, signing, and verification for RSA and ECDSA and provides measured performance values for comparison.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation